

Automated Pull Request Reviewer

May 8, 2026

Business Understanding

Business Problem

Modern software engineering teams invest substantial time in pull request (PR) review. Senior developers and tech leads are responsible for catching correctness bugs, security vulnerabilities, data-loss risks, and maintainability problems before code reaches production. Industry estimates suggest that senior developers spend **8–12 hours per week** on code review work [1], a significant fraction of which is spent on repetitive, low-level issues that a system could detect automatically.

Three structural problems motivate this project:

- **High opportunity cost:** Senior engineering time is among the most expensive organizational resources. Repeatedly catching the same categories of issues consumes capacity that could be spent on design, mentoring, and complex problem-solving.
- **Feedback latency:** Developers may wait hours or days for comments that an automated system could surface immediately. Engineering benchmarks report typical time-to-merge values in the range of **73–141 hours** [2], and a significant share of that wait is caused by review queue time.
- **Data privacy risk:** Existing AI-based code review tools require sending proprietary source code to external APIs, creating compliance and confidentiality risks that make them impractical in enterprise environments.

The goal of the **Automated Pull Request Reviewer (APR)** is to act as a privacy-first pre-review quality gate inside the CI/CD pipeline. It analyzes Python pull requests locally on company infrastructure, flags potentially blocking issues before human review begins, and generates inline comments explaining the detected issue.

Stakeholders

- **Primary: Senior developers and tech leads.** Direct beneficiaries of reduced repetitive review work. APR surfaces blocking issues before their review round, reducing the number of review cycles and the total time they spend on routine comments.
- **Secondary: Engineering managers and organizations.** Benefit from improved delivery throughput, lower Change Failure Rate, and an on-premise inference design

that satisfies data privacy and compliance requirements.

- **Indirect: All developers submitting pull requests.** Benefit from faster automated feedback and fewer repeated change requests.

Success Criteria

Success is defined at three levels. The first level captures the intended business impact of APR on the pull request process:

Table 1: Business success criteria for APR.

Business Goal	Metric	Definition	Target
Decrease PR cycle time	Time-to-Merge	Time from PR creation to merge	−20%
Improve review readiness	First-Time Approval Rate	Share of PRs approved without change requests	+5%
Reduce production risk	Change Failure Rate	Share of deployments requiring immediate remediation	−5% rel.
Reduce review noise	Comments per PR	Average human review comments per PR	−7%
Preserve code privacy	Local inference	Proprietary code must not leave secure infrastructure	Mandatory

The second level captures the offline machine learning success criteria used to evaluate model candidates before deployment:

Table 2: Offline evaluation targets for the review model.

Metric	Target	Rationale
Precision (blocking class)	> 70%	Minimize false positives and developer trust erosion
Recall (blocking class)	> 40%	Catch a meaningful share of serious issues
Localization quality	IoU > 0.55	Check whether predicted line spans match actual issue locations
Comment quality	BERTScore F1 > 0.85	Estimate usefulness of generated review comments

Precision for the blocking class is the most important offline metric. A system that posts many incorrect warnings will quickly lose developer trust, undermining the business objectives even if recall is high.

A third layer of success criteria governs APR as an operational service rather than as a model artifact. Because APR is invoked inside the pull request workflow, latency, availability, and

reliability targets must be agreed upfront so that downstream phases can measure compliance rather than redefine the bar. Table 3 states these operational success criteria; they apply uniformly to the MVP shadow deployment and to any subsequent production rollout.

Table 3: Operational success criteria for APR as a developer-facing service.

Dimension		Target	Definition / rationale
End-to-end (P50)	latency	≤ 8 s per file	Matches the order of magnitude of an interactive review action; longer responses are perceived as a workflow interruption.
End-to-end (P95)	latency	≤ 30 s per file	Long-tail bound for large or context-heavy files; violations are logged but not retried.
Availability (monthly)		$\geq 99.0\%$	Two nines are sufficient for a non-blocking advisor; equivalent to ≤ 7.2 h of unplanned downtime per month.
Successful response rate		$\geq 98\%$	Share of requests returning a schema-conforming JSON payload; covers HTTP errors, model timeouts, and parse failures jointly.
JSON parse failure rate		$\leq 1\%$	Tracked separately because parse failures degrade silently and erode developer trust faster than HTTP errors.
Throughput (sustained)		≥ 10 PRs / minute	MVP capacity envelope on a single A100 80 GB; loads above this trigger a queue-depth alert.

These targets are formal SLOs — internal commitments that define when the system is considered healthy — rather than external SLAs, since APR is not on the critical merge path during the MVP. SLA-level commitments would be established only if APR were promoted to a blocking quality gate.

Assumptions

- **Python-only scope:** For the MVP, only Python `.py` files are reviewed. Jupyter notebooks, generated files, vendored code, and non-Python files are excluded.
- **Linter separation:** Formatting and style issues (spacing, naming conventions, import ordering) are assumed to be handled by existing linters such as `black` and `flake8`. APR targets blocking semantic issues only.
- **Context window sufficiency:** The relevant context for a review decision can be compressed and selected to fit within approximately 8 000–16 000 tokens per request, covering the changed file, its direct dependencies, and repository-level metadata.
- **Label quality via LLM-as-a-judge:** Although raw human review comments are

noisy, an LLM-based labeling pipeline can classify them into a structured taxonomy with sufficient precision to produce a clean training dataset.

- **Public repository representativeness:** Public GitHub repositories are assumed to be a sufficiently representative source for training data, even though enterprise codebases may have different patterns and internal conventions.

Risks

- **Noisy training labels:** Human PR review comments include questions, suggestions, style notes, praise, and non-blocking feedback. If not cleaned effectively, the model will learn to flag non-blocking content, inflating the false positive rate.
- **Developer trust erosion:** False positive comments are disproportionately costly. A single wave of incorrect bot warnings can permanently reduce developer confidence in the tool, making it counterproductive even if precision improves in a later version.
- **LLM labeling bias:** The LLM-as-a-judge pipeline may introduce systematic label bias if the labeling model has consistent blind spots or overweights certain issue categories.
- **Distribution shift:** Training data is sourced from public GitHub repositories. Enterprise codebases may have different patterns, internal libraries, and review conventions, reducing model generalization to private environments.
- **Context window insufficiency:** For large files or deeply interconnected code, compressed context may omit the dependencies needed to detect certain bug classes, producing false negatives that are invisible in offline evaluation.

Preliminary Cost-Benefit Analysis

Projected Benefits

The primary source of value is the reduction of repetitive review effort. At the target of a **7% reduction in review noise**, a senior developer spending 8–12 hours per week on reviews saves approximately 35–50 minutes per week. At a senior developer salary of \$200 000/year, this corresponds to **\$3 000–\$4 200 per developer per year** [1].

A secondary benefit is faster delivery. A **20% reduction in time-to-merge** from a baseline of 73–141 hours [2] saves 14.6–28.2 hours of waiting time per pull request, improving engineering throughput.

If APR reduces the Change Failure Rate by **5% relative** from a baseline CFR of 5–17% [3], this avoids approximately one production failure per **115–400 pull requests**, reducing incident response and hotfix costs.

Estimated Costs

- **Labor:** Five team members, each contributing 15–25 hours. At \$16/hour, total labor cost is bounded between **\$1 200 and \$2 000**.
- **GPU Infrastructure (training):** Model fine-tuning runs on a university-provided **NVIDIA A100 80 GB** GPU. Marginal cost is \$0; at market cloud rates (\$2–3/hour for an A100), the full training campaign (approximately 30–50 GPU-hours including hyperparameter trials, retries, and validation runs) would cost **\$60–\$150**.
- **API Infrastructure (annotation):** The computing costs for the LLM-as-a-judge

pipeline were fully covered by a Yandex Cloud grant. Marginal cost is \$0; at market rates the same annotation campaign would cost approximately **\$100–\$300**.

- **GPU Infrastructure (inference):** The deployed model fits on a 16 GB consumer GPU. For an on-premise MVP serving up to ten developers, a single GPU server (RTX 4080-class card plus host machine) costs approximately **\$1 500–\$2 500** one-time, with electricity costs below \$50/month.

Return on Investment

At a one-time cost of \$3 000–\$5 000 and a conservative benefit of \$3 000–\$4 200 per developer per year (\$250–\$350/month), the system reaches break-even for a **single developer in approximately 8–20 months**. For a team of ten senior reviewers the combined monthly benefit reaches \$2 500–\$3 500, recovering the full project cost in **1–2 months**. The annualized benefit for ten developers (\$30 000–\$42 000) against the one-time project cost yields a first-year ROI of approximately **600%–1 400%**. This analysis justifies proceeding with the modeling phase.

Data Mining Tasks

The core data mining problem is formulated as a supervised binary classification task at the changed-file or diff-hunk level:

Given a Python code change and its surrounding context, predict whether the change contains a blocking issue.

Three related tasks are required for the full review pipeline:

1. **Blocking issue detection:** Binary classification that decides whether the change should be flagged. This is the primary task.
2. **Line localization:** If an issue is detected, identify the affected line or line range inside the diff to enable inline pull request comments.
3. **Comment generation:** If an issue is detected, generate a concise human-readable explanation that can be shown to the developer as an inline review comment.

Project Plan

The project follows the CRISP-DM methodology across six sequential phases.

Table 4: Project timeline with key deliverables and phase dependencies.

Phase	Main Activities	Key Deliverable	Duration
1. Business Understanding	Define problem scope, stakeholders, success criteria	Problem definition and metric targets	1 week
2. Data Understanding	EDA on GH Archive review comments; feasibility check for context length and class balance	EDA report; go/no-go on data volume	1–2 weeks
3. Data Preparation	Filter, enrich via GitHub APIs, compress context, LLM-label, construct train/val/test splits	Clean labeled dataset with PR-level split isolation	2–3 weeks
4. Modeling	Zero-shot baseline (Qwen3-1.7B), structured output parsing, fine-tuning, validation comparison	Validation metrics; fine-tuned model checkpoint	2 weeks
5. Evaluation	Precision/recall/F1, localization IoU, comment quality via BERTScore, error analysis	Final evaluation table; MVP readiness decision	2 weeks
6. Deployment and Demo	Web UI accepting a GitHub PR URL, local inference, inline comment display	Working demo, API spec, video demo	2 weeks

CRISP-DM Mapping Table

Table 5: CRISP-DM phases, project activities, and key deliverables.

Phase	Project Activities	Key Deliverable
1. Business Understanding	Define the PR review problem; identify stakeholders; set business KPIs and offline metrics; assess risks and constraints	Problem definition, metric targets
2. Data Understanding	Explore GH Archive PR review events; analyze comment volume, class balance, and context lengths; verify hardware feasibility	EDA report, feasibility check
3. Data Preparation	Filter Python PR comments; enrich via GitHub APIs; reconstruct file context; resolve dependencies; compress context; LLM-label; split by PR and repository	Labeled dataset, model-ready prompts
4. Modeling	Zero-shot baseline with Qwen3-1.7B; structured output parsing; supervised fine-tuning on training set; comparison on validation set	Baseline and fine-tuned models; validation metrics
5. Evaluation	Precision, recall, F1; localization IoU; BERTScore for generated comments; qualitative error analysis	Final evaluation table, error analysis
6. Deployment	Build web UI accepting a GitHub PR URL; local inference pipeline; structured JSON response format; video demo	Demo application, API documentation

Data Understanding

Collect Initial Data

The dataset was sourced from the **GH Archive** public event stream, which provides a continuous log of all public GitHub events. The raw data consists primarily of review comment events for Python repositories downloaded over a fixed time range. To complement these events, additional data parts were acquired from the GitHub GraphQL API (for pull request metadata) and the GitHub REST API (for patched code, repository files, and dependency trees). In total, the collected raw data covers **1,404 pull requests**, **1,599 snapshot commits**, **6,597 changed files**, and **9,841 review comments**.

Describe Data

The Parquet export is organized at the file level: each row corresponds to one changed Python file within one snapshot commit of a pull request. The schema is described in Table 6.

Table 6: Schema of the exported dataset (`dataset.parquet`).

Column	Type	Description
<code>pr_title</code>	string	Pull request title
<code>pr_body</code>	string	Pull request description in Markdown or HTML
<code>repo_star_count</code>	integer	Repository star count at collection time
<code>patched_content</code>	string	Full content of the changed file at the snapshot commit
<code>metadata_files</code>	list[dict]	Repository-level context files (README.md, requirements.txt, setup.py, pyproject.toml, setup.cfg), each as a <code>{path, content}</code> record
<code>file_tree</code>	string	Top-level directory listing of the repository at the snapshot commit
<code>incoming_dependencies</code>	list[dict]	Files that import identifiers changed in the main file, each as a <code>{path, content}</code> record
<code>outgoing_dependencies</code>	list[dict]	Files imported by the main file, each as a <code>{path, content}</code> record
<code>comments</code>	list[dict]	Review comments left on this file in the pull request

The unit of analysis aligns with the modeling task: each row represents a self-contained code review decision context, combining a changed file with its surrounding repository and PR metadata.

Explore Data

Comment Distribution

Of the 6,597 changed files, **3,781 (57.3%)** carry at least one review comment, while **2,816 (42.7%)** have no comments. In total, 9,841 review comments were collected.

Token Length Analysis

Since the planned models are large language models with a finite context window, the feasibility of including all input components in a single inference pass must be assessed. Token lengths were estimated using the heuristic of 1 token $\approx$ 4 characters. Table 7 summarizes the key percentiles for each component, including dependency content aggregated per main file.

Table 7: Estimated token length percentiles per input component.

Component	Median	75th pct	95th pct
PR metadata (title + body)	135	315	1,094
README files	550	1,167	3,428
Configuration files	241	590	1,629
Repository file tree	2,376	9,616	109,696
Main changed file	1,709	4,295	15,373
Incoming dependencies	0	2,455	43,768
Outgoing dependencies	2,150	13,618	59,186

All distributions are strongly right-skewed, with most entries concentrated at low token counts but long tails. Concatenating all components without truncation is therefore infeasible for a non-trivial fraction of examples, confirming the necessity of selective extraction and truncation in the Data Preparation step.

Dependency Coverage

Dependency enrichment succeeded for a large share of files: **64.6%** have at least one outgoing dependency (21,011 total entries across the dataset) and **34.7%** have at least one incoming dependency (15,538 total entries). The per-file count distributions are summarized in Table 8.

Table 8: Distribution of per-file dependency counts.

Dependency type	Mean	Std	Median	75th pct	Max
Outgoing	3.41	5.87	2	4	116
Incoming	2.54	13.78	0	1	460

The extreme maximum values (116 outgoing, 460 incoming) combined with the heavy token-length tails shown in Table 7 make it infeasible to include all dependency files verbatim. However, the 75th percentile is only 4 for outgoing and 1 for incoming dependencies, so retaining the top 5 most relevant files per type preserves complete coverage for the majority of examples while eliminating the extreme right-tail cases.

Qualitative Example Analysis

To complement the quantitative distributions, a random sample of three instances per content type was manually inspected. The following observations informed the Data Preparation strategy:

- **PR bodies:** The opening paragraph or bullet list consistently captures the intent of the change; everything beyond is checklists, boilerplate, and HTML markup from contributor templates.
- **README files:** The first paragraph provides the project synopsis; subsequent sections (installation, contributing, badges) are irrelevant to code review and inflate character counts substantially.

- **Configuration files:** Files such as `pyproject.toml` mix compact, informative metadata (name, description, dependencies) with linting and build-system configuration that carries no review signal; regex extraction of the metadata block is sufficient.
- **Repository file trees:** Dot-directories (`.git`, `.github`, `.vscode`) are ubiquitous but uninformative; the meaningful structure is fully captured within two levels of depth.

Verify Data Quality

- **Missing Values:** The absence of review comments (2,816 files, 42.7%) represents valid data indicating no code review activity on a given file, not a collection failure. Repository metadata files were successfully retrieved for **94.5%** of snapshot commits and directory trees for **95.4%**. The remaining 5.5% and 4.6%, respectively, correspond to repositories that were deleted or made private after collection; these rows will be removed in the Data Preparation step. In contrast, rows with empty lists for incoming or outgoing dependencies are not considered quality failures, as some files naturally have no dependencies. The core review signal in `patched_content` remains intact for these cases.
- **Duplicate Detection:** Exact duplicate rows were identified on the composite key (`pr_url`, `file_path`, `snapshot_commit_sha`). A total of **194 duplicate entries (2.9%)** were found, produced by overlapping GH Archive hourly dump boundaries, leaving 6,403 unique rows. These duplicates will need to be removed in the Data Preparation step.
- **Data Consistency:** The `patched_content` field reflects the file state at the snapshot commit rather than the final merged state; this is intentional by design, as the model must reproduce the information available to a human reviewer at review time, not retrospective knowledge of the final merge outcome.

Data Preparation

Select Data

We selected only the rows representing Python `.py` files from the integrated dataset, discarding non-Python files as they fall outside the project scope. This granularity matches the real-world reviewer workflow: a reviewer examines one file at a time and leaves comments anchored to specific lines. All fields from the enriched Parquet described in Table 6 are carried forward.

Clean Data

The raw GH Archive event stream is filtered through a preprocessing pipeline before the Parquet export, and a further token-budget filter is applied during LLM annotation. The full set of filtering decisions is described in Table 9.

Table 9: Data cleaning filters and their methodological purpose.

Filter	Purpose
.py files only	Enforces the Python-only scope assumption from Business Understanding.
Left-side diff comments excluded	Comments anchored to removed (left-side) diff lines refer to code that no longer exists in the PR and cannot be cross-referenced with the current patch.
Reply comments excluded	Threaded reply comments are reactions to earlier review messages, not independent assessments of the code; including them would duplicate signals and introduce contextless noise.
Failed enrichment dropped	Pull requests for which the base commit or patch could not be retrieved via the GitHub API are dropped entirely.
Token-length cap ($\leq 30,000$ tokens)	Prompts exceeding this limit are skipped during LLM judge annotation.

After filtering, the dataset contains **3,521 files with at least one review comment** and **2,490 with no comments**, for a total of **6,011 samples** before label construction.

Construct Data

Comment Taxonomy Development

Raw PR comments are highly heterogeneous: they include bug reports, style suggestions, questions, praise, and off-topic remarks. A structured taxonomy is required to distinguish blocking issues from non-blocking feedback. Approximately 500 comments were sampled and provided to DeepSeek-v3 to inductively derive a set of mutually exclusive categories. The resulting taxonomy of nine categories and their observed distribution across the 7,190 annotated comments are shown in Table 10.

Table 10: LLM-derived comment taxonomy with observed class distribution.

Category	Semantics	Count
<code>blocking_issue</code>	Correctness bugs, security flaws, and crash-inducing errors	721
<code>performance</code>	Algorithmic or resource-efficiency concerns	99
<code>best_practice</code>	Design patterns, idiomatic Python, and structural improvements	2,295
<code>style</code>	Formatting, naming conventions, and cosmetic changes	1,754
<code>documentation</code>	Missing or incorrect docstrings and type hints	632
<code>question</code>	Requests for clarification from the reviewer	885
<code>nitpick</code>	Minor observations with negligible impact	682
<code>praise</code>	Positive feedback from the reviewer	88
<code>other</code>	Uncategorized or ambiguous comments	34

Context Compression Pipeline

The EDA showed that raw token lengths are strongly right-skewed; concatenating all fields verbatim would exceed the 16,384-token training budget for a significant fraction of examples. A deterministic compression pipeline — using no summarization LLM — is applied to ensure full reproducibility.

Field-level truncation rules are applied first to bound each input component:

- **`pr_body`:** HTML stripped and truncated to 500 characters.
- **`metadata_files`:** README capped at 400 characters, CONTRIBUTING at 400 characters, and `setup.py` at 500 characters; total metadata capped at 4,000 characters.
- **`file_tree`:** Truncated to two levels of depth; deeper levels are uninformative per the qualitative analysis in Data Understanding.
- **`outgoing_dependencies` and `incoming_dependencies`:** Top-5 most-relevant files per type retained; boilerplate files (`__init__.py`, `setup.py`, `conftest.py`) excluded.

Diff-level compressions are then applied to `patched_content` to collapse redundant structure without losing semantically important lines:

- Unchanged function bodies collapsed to `[function body is hidden]`.
- Multi-line docstrings longer than three lines replaced with `[docstring hidden]`.
- Comment blocks longer than three lines condensed to the first three lines plus a summary marker.
- Excessive blank lines reduced to a maximum of two consecutive blank lines.

After compression, **77.2%** of samples fit within the 16,384-token training budget, with a median assembled length of approximately 6,700 tokens.

LLM-as-Judge Annotation

The compressed context for each sample is passed to a **Yandex Cloud Qwen 3 235B** judge model. Two prompt variants are used depending on whether the file carries human comments:

- **Classification prompt** (files with comments): each comment is assigned exactly one category from the taxonomy in Table 10.
- **Validation prompt** (no-comment files): the model answers a binary question — does this file contain a blocking issue? — to confirm that candidate negatives are genuinely clean.

Of the 2,490 no-comment candidates, **1,897 were validated as clean** (label-eligible negatives), **305 were flagged as containing blocking issues** and removed from the negative pool, and 288 were skipped for exceeding the token budget.

Binary Label Construction

The final binary label is defined conservatively to maintain high precision, in line with the >70% precision target established in Business Understanding. The label assignment rule is:

- **Label = 1 (positive)**: Files where the judge assigned **blocking_issue** (100% included), **performance** (100% included), or **best_practice** (50% random sample).
- **Label = 0 (negative)**: Files with no review comments that were validated as clean by the judge.

Although the raw comment counts in the taxonomy are higher, consolidating these comments at the file level and applying the token-length and cleaning filters defined in Table 9 yields a final set of **1,067 positive examples** and **1,897 negative examples**. Categories such as **style**, **documentation**, **question**, **nitpick**, and **praise** are excluded from training labels and preserved for future hard-negative experiments.

Integrate Data

The full dataset was assembled by integrating the raw event stream with GitHub API data through a four-stage pipeline:

1. **GH Archive fetch**: Review comment events were extracted for the selected time range to identify relevant pull requests.
2. **GraphQL enrichment**: The PR metadata (title, body, repository star count) was joined to the corresponding PR event using the GitHub GraphQL API.
3. **Code enrichment**: For each changed Python file, the full patched content was reconstructed from base and snapshot zipballs. Two types of contextual dependencies were resolved: *outgoing dependencies* (files that the changed file imports) and *incoming dependencies* (files elsewhere in the codebase that import identifiers defined in the changed file). Both sets were computed by parsing Python import graphs against the resolved symbol diff.
4. **Repository context enrichment**: Repository-level context files (README, configuration files) and the directory tree were extracted from the zipballs and appended to

the relevant file records.

The integrated records were subsequently flattened into a row-per-file Parquet dataset.

Format Data

Each sample is serialized as a single JSON object and written to a line-delimited JSONL file. The schema contains three field groups:

- **Training fields:** `prompt` (the assembled and compressed context string ready for model input), `target` (a JSON object listing detected issues with line ranges and explanations, or an empty list for negatives), and `label` (0 or 1).
- **Raw context:** `patched_content`, `outgoing_dependencies`, `incoming_dependencies`, `metadata_files`, `file_tree`, and `comments` - retained for debugging, re-labeling, and ablation experiments but not fed to the model during training.
- **Provenance fields:** `comment_categories`, `has_blocking_issues`, `llm_raw_response`, and `split` - enable full reproduction of the label construction pipeline from raw data.

Modeling

Select Modeling Technique

The APR task combines three related outputs — blocking issue detection, line localization, and review comment generation — that all require the same contextual reasoning over a code diff. To avoid error propagation between pipeline stages, all three outputs are produced in a **single structured-generation pass**: the model receives the assembled review prompt and returns a JSON object conforming to the schema below.

```
{
  "issues": [
    {
      "line_range": {
        "start": <int>,
        "end": <int>
      },
      "comment": "<string>"
    }
  ]
}
```

An empty `issues` list encodes the negative class (no blocking issue detected). This framing means that the binary detection decision is implicit in whether the list is non-empty, eliminating the need for a separate classifier head and keeping the pipeline to a single inference call per file.

The task is therefore formulated as **supervised sequence-to-sequence generation** with a structured output target. The model family chosen is **Qwen3-1.7B**, selected on the basis

of LiveCodeBench Pass@1 scores among models in the 1.3B–2B parameter range that fit the available hardware (Table 11). LiveCodeBench is used as a contamination-free proxy for general code-understanding ability; higher Pass@1 correlates with better diff reasoning and structured output compliance.

Table 11: Candidate models evaluated for baseline selection.

Model	Params	Context	LCB Pass@1	Decision
Qwen3-1.7B	1.7B	32,768	33.2	Selected
OpenCoder-1.5B-Instruct	1.5B	4,096	12.8	Alt. baseline
DeepSeek-Coder-1.3B-Instruct	1.3B	16,384	5.1	Secondary
Yi-Coder-1.5B-Chat	1.5B	128,000	4.8	Secondary
CodeLlama-7B-Instruct	7.0B	16,384	7.1	Too large

Two modeling approaches are evaluated against the same test set:

1. **Zero-shot baseline:** Qwen3-1.7B with only prompt engineering; no gradient updates.
2. **Supervised fine-tuning (SFT):** The same model trained on the labeled dataset from Data Preparation.

Generate Test Design

Dataset split. The final labeled dataset contains **2,964 samples**: 1,067 positive examples and 1,897 negative examples. Since data leakage is especially dangerous in code-review tasks, the split is constructed using a multi-level isolation strategy rather than a simple random split.

First, the **test set** is constructed from pull requests that contain at least one confirmed **blocking_issue**. We select 100 such pull requests and include all eligible changed files from them: positive files with confirmed blocking issues and negative files from the same PRs that were validated as clean by the LLM judge. This produces a test set of **310 samples**: 140 positives and 170 negatives.

Second, to prevent repository-level leakage, every repository that appears in the test set is removed completely from the remaining training and validation pool. In total, this repository holdout step removes **96 repositories**. This ensures that the final test metrics measure generalization to unseen projects rather than memorization of repository-specific conventions, APIs, or coding style.

Third, the remaining samples are split into training and validation subsets with stratification by label. The resulting training set contains **1,913 samples** and the validation set contains **536 samples**. The validation set is used exclusively for checkpoint selection during fine-tuning, while the test set is held out until final evaluation.

Table 12: Final train/validation/test split after PR-level isolation and repository-level test holdout.

Split	Samples	Positives	Negatives	Positive Share
Train	1,913	687	1,226	35.9%
Validation	536	162	374	30.2%
Test	310	140	170	45.2%

The test set has a higher positive share than the training and validation sets because it is intentionally sampled from pull requests containing at least one blocking issue. This design makes the final evaluation more informative for the target use case: detecting blocking issues in PRs where such issues are realistically present. Since the reported metrics include precision, recall, and F1-score for the blocking class, the evaluation remains meaningful even though the test class prior is not identical to the training distribution.

Evaluation metrics. Model performance is evaluated against the offline machine learning targets defined previously in Table 2, specifically prioritizing precision and localization quality.

Build Model

Zero-Shot Baseline

The baseline uses Qwen3-1.7B loaded in `bfloat16`. The review prompt is assembled using the context compression pipeline described in the Construct Data section. The model is explicitly instructed to return only valid JSON.

Generation uses deterministic greedy decoding: `do_sample = False`, `max_new_tokens = 512`, repetition penalty = 1.0 (disabled). Responses are parsed with `json.loads`; any response that fails to parse is treated as an empty-issues prediction (label = 0), reflecting the conservative precision-first evaluation philosophy.

Supervised Fine-Tuning

Fine-tuning uses the HuggingFace `Trainer` with the hyperparameters listed in Table 13. Key design choices are described below.

Table 13: Fine-tuning configuration.

Hyperparameter	Value	Rationale
Base model	Qwen/Qwen3-1.7B	Highest LiveCodeBench score in target size range
Max sequence length	16,384 tokens	Capped to fit GPU memory; covers $\approx 77\%$ of compressed samples
Per-device batch size	1	Required by long sequences on A100 80 GB
Gradient accumulation steps	8	Effective batch size of 8
Learning rate	2×10^{-5}	Standard SFT range for 1–2B models
Training epochs	3	Convergence verified on validation loss
Warmup ratio	0.03	3% of total steps
Weight decay	0.01	L2 regularization
Gradient norm clip	1.0	Prevents loss spikes on long sequences
Floating-point precision	BF16	Numerical stability; native on A100
Gradient checkpointing	Enabled	Reduces activation memory $\approx 30\%$

Loss masking. Each sample is wrapped with Qwen’s chat template via `apply_chat_template()`, producing a prompt (user role) and a JSON target (assistant role). Labels for the prompt tokens are set to -100 (masked from the cross-entropy loss); the loss is computed only over the assistant span. This prevents the model from overfitting to prompt boilerplate and focuses learning entirely on the structured output schema.

Prompt suffix fitting. When a serialized sample exceeds 16,384 tokens, the oldest context sections (repository-level metadata, file tree) are truncated from the left of the prompt while the diff and target are always preserved in full. This retains the most decision-relevant content within the hardware budget.

Checkpoint selection. The model is checkpointed every 500 training steps; the three best checkpoints by validation loss are retained. The checkpoint with the lowest validation loss is used for final evaluation.

Assess Model

Both models are evaluated on the same 310-sample test set under the identical pipeline. Table 14 reports all metrics side by side.

Table 14: Zero-shot baseline vs. fine-tuned model on the test set

Metric	Baseline	Fine-Tuned	Target
Precision (blocking class)	43.3%	74.8%	>70% ✓
Recall (blocking class)	18.6%	55.0%	>40% ✓
F1 (blocking class)	0.260	0.634	-
Accuracy	52.3%	71.3%	-
TP / FP / TN / FN	26 / 34 / 136 / 114	77 / 26 / 144 / 63	-
Mean line-range IoU	0.450	0.612	>0.55 ✓
BERTScore F1 (aligned pairs)	0.838	0.869	>0.85 ✓

Zero-Shot Baseline Results

The zero-shot model (Table 14, Baseline column) is conservative: precision (43.3%) exceeds recall (18.6%) by a wide margin, indicating that the model predicts a blocking issue only when the signal is very strong. Most true positives (114 out of 140) are missed. The IoU of 0.450 shows that, when the model does predict an issue, the line spans are moderately well calibrated even without training. Neither the precision nor the recall targets are met by the zero-shot model alone.

Fine-Tuned Model Results

The fine-tuned model (Table 14, Fine-Tuned column) meets both primary success criteria. Precision rises from 43.3% to 74.8%, indicating that developers will receive relatively few false-positive warnings. Recall rises from 18.6% to 55.0%.

Line localization improves from IoU 0.450 to 0.612, likely because the model learns to anchor its predictions to the explicitly numbered lines provided in the prompt. BERTScore on IoU-aligned pairs improves modestly from 0.838 to 0.869, consistent with fine-tuning producing more concise and targeted comments rather than changing underlying language quality.

Error Analysis

The 63 remaining false negatives fall into two observable patterns. First, issues that span a large fraction of the file (e.g. architectural mistakes or missing exception handling that applies throughout) tend to be missed because the model’s attention is drawn to local, syntactically salient changes. Second, a subset of negatives correspond to borderline “best practice” labels that were included as positives at a 50% sampling rate during label construction; these are inherently ambiguous and may not be consistently detectable.

The 26 false positives are dominated by cases where the diff introduces legitimate new patterns (e.g. a `try/except` block) that superficially resemble common bug patterns; without access to the full business logic, the model flags them conservatively. This residual false-positive rate

(26/103 predicted positives = 25%) is within the acceptable range for the precision target and unlikely to erode developer trust at production volume.

Evaluation

Evaluate Results

The goal of this phase is to determine whether the modeling results are sufficient for the original business problem: reducing repetitive pull request review work while preserving developer trust and code privacy.

The original data mining objective was to build a local automated reviewer capable of detecting blocking issues in Python pull requests, localizing the affected line range, and generating a useful inline comment. The fine-tuned Qwen3-1.7B model meets all four offline success criteria from table 2: blocking-class precision **74.8%** (target $\geq 70\%$), recall **55.0%** (target $\geq 40\%$), mean line-range IoU **0.612** (target ≥ 0.55), and BERTScore F1 **0.869** (target ≥ 0.85). All four metrics also improve substantially over the zero-shot baseline (table 14), which confirms that supervised fine-tuning on the prepared dataset adds signal beyond prompt engineering alone.

From a business perspective, these results show that APR can act as a **pre-review quality gate**: the model is not expected to replace human reviewers, but it can detect a meaningful share of blocking issues before senior developers begin manual review, directly supporting the goals of reducing review noise and improving review readiness stated in Business Understanding. The privacy success criterion from table 1 is satisfied as well, because Qwen3-1.7B is small enough to run on a single 16 GB consumer GPU and proprietary source code does not leave secure infrastructure during inference.

The operational success criteria from table 3 were partially verified at this stage. Latency, throughput, and JSON parse failure rate are measurable directly on the test set: end-to-end inference on a single A100 in BF16 reaches a median latency of **3.2 s per file** (P95 = **9.4 s**), well within the $P50 \leq 8\text{ s}$ and $P95 \leq 30\text{ s}$ budgets; sustained throughput is **22 PRs / minute** with vLLM continuous batching, above the 10 PRs/min target; and the fine-tuned model returns a schema-conforming JSON payload on **99.6%** of predictions, below the 1% parse-failure budget. Availability and monthly successful response rate cannot be measured offline by construction and are deferred to the deployment phase as production observability targets.

At the same time, the offline result has clear limits. The headline precision of **74.8%** is computed on a test set deliberately enriched with PRs containing blocking issues, where positive prevalence is 45.2%. In real PR traffic, where only 10–15% of PRs typically contain blocking issues, this same precision drops to approximately **28–39%**. This is an expected effect of the enriched test set and will be addressed during shadow-mode rollout by tightening the model’s confidence threshold for posting a comment. Beyond this, the model is evaluated only on public GitHub repositories, while enterprise codebases may have different patterns and review conventions; labels are partially derived through an LLM-as-a-judge pipeline, which can introduce systematic bias on borderline cases; and the broader business KPIs from table 1 – a 20% reduction in Time-to-Merge, a 5% increase in First-Time Approval Rate, a 5%

relative reduction in Change Failure Rate, and a 7% reduction in human review comments per PR – are not measurable on a static dataset and require shadow-mode or production evaluation.

Overall, the result is **successful as an offline data mining prototype**. The fine-tuned model meets every offline machine learning success criterion and clearly outperforms the zero-shot baseline. From the business perspective the result is **promising but not yet fully validated**: APR is ready for controlled shadow-mode testing, but not yet ready for unsupervised production deployment or autonomous blocking of pull requests.

Review Process

This subsection reviews the CRISP-DM process that was followed and identifies which decisions were well supported and which parts should be strengthened in the next iteration.

1. Business Understanding. The Business Understanding phase prevented the project from being framed as a generic code-generation exercise and tied APR to a concrete operational problem with a clear precision-first priority. It also defined operational SLOs upfront in table 3, which made the deployment targets verifiable rather than negotiable. The unresolved part of this stage is that several business KPIs (Time-to-Merge, First-Time Approval Rate, Change Failure Rate) are by construction not measurable until the system is integrated into a real review workflow.

2. Data Understanding. The Data Understanding phase converted modeling assumptions into measurable constraints rather than only describing the dataset: the right-tailed token distributions of repository trees and dependency files directly motivated the context-compression pipeline, and the dependency analysis showed that retaining the top-5 incoming and outgoing files preserves coverage for most examples while keeping prompt length manageable. The token-length estimation relied on the standard “1 token $\approx$ 4 characters” heuristic; using the actual Qwen tokenizer would have been more accurate for code, and this should be tightened in the next iteration.

3. Data Preparation. The Data Preparation phase was the most important bridge between raw GitHub data and the modeling setup. Filtering, GitHub API enrichment, and the context-compression pipeline produced a coherent JSONL training format with both prompts and structured JSON targets. The main weakness of this stage is label reliability: labels were constructed through an LLM-as-a-judge pipeline rather than full manual annotation, and the positive class consolidates `blocking_issue`, `performance`, and a 50% sample of `best_practice`. This dilutes the strict definition of “blocking” used in the precision target and is the most likely source of borderline disagreements between labels and human reviewer intent. A future iteration should keep these categories separate and report per-category metrics.

4. Modeling. The model was trained to produce a structured JSON response covering detection, line range, and comment text in a single inference call, which matches the intended product behavior. The comparison with the zero-shot baseline confirmed that fine-tuning was necessary: precision improved from **43.3%** to **74.8%**, recall from **18.6%** to **55.0%**, and IoU from **0.450** to **0.612**. The main weakness of this stage is that the headline numbers come from a single training seed and a single test split; a repeat of the project should report seeds

variance and include at least one strong non-LLM baseline (for example a linter pipeline) to anchor the comparison.

5. Evaluation Method. Evaluation covered all three technical outputs (detection, localization, comment quality) and verified compliance with the offline ML criteria and the offline-measurable subset of the operational SLOs. The methodological gap is that the test set is enriched with PRs containing blocking issues, which makes the precision figure optimistic relative to expected production prevalence; this should have been mitigated either by also reporting precision under expected production prevalence or by including a separate held-out random sample. The business KPIs were correctly identified as not measurable offline and deferred to deployment.

6. Review Summary. Overall, the process produced a coherent offline APR prototype and validated the fine-tuned model as a technical component. The main unresolved issues are label reliability, distribution shift between public GitHub and enterprise repositories, the test-set construction bias on the precision figure, and the lack of validation in a real pull request workflow. None of these block shadow-mode deployment, but each must be addressed before any conclusion about production readiness.

Determine Next Steps

Based on the current results, the project should continue to a controlled MVP deployment rather than stop at offline evaluation. The fine-tuned model meets the predefined offline success criteria, so the next step is to test whether these gains translate into measurable improvements in the real pull request workflow.

The model should be deployed as an internal APR service integrated into the pull request process. To limit risk, the first deployment should be restricted to Python pull requests and should run in shadow mode followed by a controlled A/B experiment. In the treatment group APR analyzes pull requests before or during human review and surfaces automated comments; in the control group pull requests follow the normal review process without APR assistance.

The A/B experiment should use the business success criteria from table 1 as its primary decision metrics, comparing Time-to-Merge, First-Time Approval Rate, Change Failure Rate, and Comments per PR between treatment and control. The operational SLOs from table 3 must continue to be met throughout the experiment, with sustained breach over a rolling 7-day window triggering rollback. Developer trust should also be tracked through APR comment acceptance rate and false-positive reports, since trust is the main practical risk behind the precision-first offline target.

Two improvements should feed the next CRISP-DM iteration in parallel with the deployment work. First, label quality: separating `blocking_issue`, `performance`, and `best_practice` into distinct training labels and re-measuring per-category precision will clarify whether the 74.8% figure holds for blocking issues in the strict sense. Second, distribution validation: at least one private repository should be added to evaluation to estimate the precision drop expected when moving from public GitHub to enterprise code patterns.

The recommendation is therefore to deploy the current model as a limited MVP in shadow mode, evaluate it through an A/B experiment with the business and operational targets

defined upfront, and use the resulting evidence—together with the label and distribution improvements—as input to the next CRISP-DM iteration. If APR improves the business metrics without reducing developer trust, it can be expanded to more repositories and integrated more deeply into the review workflow. If the experiment shows weak or negative impact, the next iteration should focus on label quality, false-positive reduction, and adaptation to enterprise-specific code patterns.

Deployment

Plan Deployment

The deployment goal is to make the fine-tuned APR model usable in a realistic pull request workflow. At this stage, the system should be deployed as a controlled MVP, not as a fully autonomous production reviewer.

The fine-tuned model will be exported as a reusable checkpoint and deployed on a GPU-enabled machine. The inference service will be containerized and built around a **vLLM + FastAPI** backend. vLLM will serve the model efficiently and support concurrent requests, while FastAPI will expose a stable API endpoint for the rest of the application. This design also keeps the model replaceable: future fine-tuned checkpoints can be deployed behind the same API. The stack on a single A100 80GB is sized to meet the operational service-level objectives defined in table 3; sustained breach of any SLO over a rolling 7-day window triggers a rollback to the previous stable model and prompt version. APR is not on the critical merge path at the MVP stage, so external SLAs are not committed to.

For the current MVP, the user-facing layer is a **Streamlit demo interface**. Developers and reviewers can submit a GitHub pull request URL; the application then runs the pipeline described earlier in the project — collect PR metadata, reconstruct changed Python files, add repository and dependency context, compress the input, assemble the prompt, run inference, and parse the structured JSON response — and renders the result as review-style comments anchored to the predicted line ranges. The Streamlit interface is intended only for demonstration and operator-driven inspection.

The next deployment step is integration with the customer’s internal Version Control System — for example as a **GitHub App** subscribed to PR events, or as an equivalent webhook on GitLab or Bitbucket. At that point, every new or updated PR is automatically processed by APR, and the comments are posted back to the PR thread through the platform’s review API. This integration is also where the A/B experiment will run: when the integration receives a PR event, it assigns the PR to the treatment or control group based on a deterministic hash of its numeric **pull request ID**, independently of repository, author, and PR size. Treatment-group PRs receive APR comments before human review; control-group PRs are silently logged but produce no PR-thread output, so the human review workflow proceeds unchanged. PR-level randomization is the smallest meaningful unit of independent observation and avoids the operational overhead of repository- or developer-level splits, while keeping enough sample size to compare the business KPIs from table 1 between the two groups. A known limitation is spillover — a developer with PRs in both groups may carry insights from APR comments seen on the treatment side into later control-group PRs — which is acceptable at the MVP

scale.

If the A/B experiment confirms positive impact on the business metrics from table 1 without degrading developer trust, the integration can be moved out of A/B mode and default to processing all PRs in the participating repositories. Wider rollout to additional repositories or to other VCS platforms is gated on a separate shadow-mode validation per platform, since PR-event semantics and review-comment APIs differ between providers.

Plan Monitoring and Maintenance

After deployment, APR is monitored continuously both as a machine learning system (for model degradation) and as a developer-facing product (for trust erosion).

The primary feedback channel is **developer-rated comments**. For each APR comment posted to a PR, the reviewer can mark it as useful, incorrect, or irrelevant; these labels are stored together with the model version, prompt version, repository, file type, predicted line range, and generated comment. The resulting log accumulates into a human-verified golden set: false positives, false negatives, and reviewer-corrected comments are the highest-value entries and feed the next training iteration. A new checkpoint replaces the current one only if it improves precision and workflow metrics on this golden set without increasing the share of negatively rated comments in subsequent shadow-mode evaluation.

System reliability is tracked through the operational SLOs defined in table 3; the breach rule and rollback procedure are the same as in Plan Deployment. The added value of the monitoring layer is correlation: every reliability incident is tagged with the model version and prompt version through the prediction log, so regressions can be attributed to a specific change rather than diagnosed in aggregate.

Workflow impact is monitored through the business KPIs from table 1. During the A/B experiment, KPIs are compared between treatment and control groups; once APR is rolled out by default, the same KPIs continue to be tracked over time so that gradual degradation (for example a creeping rise in Comments per PR) is detected before it becomes structural. APR remains a non-blocking assistant in all scenarios: automatic blocking of merges is out of scope until at least two consecutive evaluation windows show stable precision, a stable useful-comment ratio, and continued KPI improvement.

Produce Final Report

The final project deliverable consists of the CRISP-DM report, the prepared dataset schema and JSONL splits, the fine-tuned APR model checkpoint, and the MVP inference pipeline including the Streamlit operator console. Together, these artifacts document how the original pull request review problem was translated into a data mining task, how the model was trained and evaluated against both the offline ML success criteria (table 2) and the operational SLOs (table 3), and how it should be deployed for controlled MVP testing on the customer's VCS platform. All artifacts are versioned and stored alongside the report in the project repository.

Review Project

This subsection reviews the overall project execution – scope, constraints, and deliverables – rather than the data mining process itself, which is covered in Review Process.

The project executed within its planned scope. The Python-only MVP boundary set in Business Understanding held through every subsequent phase, which kept the deliverables tractable for a five-person team within the available timeline. All four CRISP-DM-aligned deliverables (labeled dataset, fine-tuned checkpoint, evaluation report with both ML and operational success criteria, and MVP inference pipeline) were completed.

Several external constraints shaped the result. The most significant was data access: the project relied on public GitHub data and an LLM-as-a-judge labeling pipeline because there was no agreement in place to collect enterprise pull request data or to fund full manual annotation. This is a documented limitation of the resulting model rather than a methodological mistake.

If the project were repeated, three setup-level changes would be highest priority. First, secure a partnership with at least one private repository owner before the data collection phase, so distribution shift can be measured rather than only acknowledged. Second, allocate a small annotation budget for human verification of borderline labels (especially the `best_practice` / `blocking_issue` boundary) rather than relying entirely on the LLM judge. Third, plan the customer-side VCS integration from the beginning, so the A/B experiment can run inside the same project iteration rather than being deferred to a follow-up.

Overall, the project should be considered a successful offline prototype rather than a finished production system. The next project iteration should focus on MVP deployment under the conditions defined in Plan Deployment, an A/B experiment in a real review workflow, and the label-quality and distribution-validation improvements identified in Determine Next Steps.

Team Contributions

Table 15: Team member contributions by project task. Percentages indicate individual share of effort; hours are average per person.

Task	Description	<i>E. Chernobrovkin</i>	<i>N. Tiurkov</i>	<i>N. Adagunov</i>	<i>I. Ershov</i>	<i>R. Gatiatullin</i>	Key Deliverables	<i>Avg. Hours</i>
Business Understanding	Problem scope, stakeholders, success criteria, cost-benefit analysis	20%	5%	30%	5%	40%	Business KPIs (Table 1); ML targets (Table 2); operational SLOs (Table 3); project plan (Table 4)	4
Data Collection	GH Archive fetch; GitHub API enrichment; patch reconstruction; Parquet export	0%	60%	30%	10%	0%	<code>dataset.parquet</code> schema in Table 6 (1,404 PRs, 6,597 files)	— 6
Data Understanding	Comment distribution, token-length and dependency coverage analysis	0%	50%	50%	0%	0%	Token-length percentiles (Table 7); dependency stats (Table 8)	4
Data Preparation	Context compression; LLM-as-judge annotation; label construction; train/val/test splits	10%	30%	0%	30%	30%	Cleaning filters (Table 9); taxonomy (Table 10); splits (Table 12)	5
Zero-Shot Baseline	Qwen3-1.7B zero-shot inference with prompt engineering	20%	0%	0%	60%	20%	Baseline column of Table 14 (P 43.3%, R 18.6%, IoU 0.450)	5
Model Fine-Tuning	Supervised fine-tuning with loss masking; checkpoint selection	20%	0%	0%	60%	20%	Model selection (Table 11); config (Table 13); fine-tuned checkpoint	6
Evaluation	Precision, recall, F1, IoU, BERTScore; SLO verification; error analysis	30%	30%	0%	20%	20%	Full results in Table 14; error analysis in text	4
Deployment & Demo	FastAPI/vLLM inference service; Streamlit demo UI	50%	10%	20%	10%	10%	Streamlit demo (GitHub PR URL input); FastAPI/vLLM service	4
Report Writing	CRISP-DM documentation	90%	0%	10%	0%	0%	This report (<code>docs/reports/report.tex</code>)	5
Task Presentation	Final project presentation slides	55%	25%	10%	0%	10%	Project slides (PDF)	4

Table 16: Group roles and overall contribution by member.

Group Member	Assigned Role(s)	Main Responsibility	Contribution
E. Chernobrovkin	Project Manager (0.7); ML Engineer (0.3)	Report coordination, modeling support, and final integration	20%
N. Tiurkov	Data Scientist (0.7); Project Manager (0.3)	Data collection, EDA, dependency analysis, and planning support	20%
N. Adagamov	Business Analyst (0.7); Data Scientist (0.3)	Business interpretation, dataset description, and analysis support	20%
I. Ershov	ML Engineer (0.7); Business Unit (0.3)	Fine-tuning, baseline implementation, and technical feasibility framing	20%
R. Gatiatullin	Business Unit (0.7); Business Analyst (0.3)	Business criteria, model support, and final recommendations	20%